

Exercises through Week 1 with Solutions

Analytics for a Better World

Do Quynh Anh Ngo
Nicole Guarnieri

Lars Boon
Joaquim Gromicho

Mats van der Vlugt
Dick den Hertog

2026/2027

Working with UvA AI Chat in ABW

UvA AI Chat is the UvA-controlled generative-AI environment available to all students with a UvAnetID. In ABW, it may act as a tutor, reviewer, or debugging partner, but it may not replace your own modelling, programming, or explanation. Open it at aichat.uva.nl.

Course rules

- **Tutorial practice:** you may use UvA AI Chat when an exercise or tutor permits it. Start from your own attempt and remain able to explain every line of code, equation, and modelling choice.
- **Graded work:** AI use is forbidden unless the assignment gives explicit permission. If it is permitted, follow the stated reporting rules and be transparent about how the tool was used.
- **Exam:** the exam is invigilated and completed entirely with pen and paper. The only permitted device is a UvA-approved calculator with no connectivity. Computers, phones, tablets, smartwatches, UvA AI Chat, and all other connected or programmable devices are not permitted.
- **Data:** do not upload personal data about other people, confidential business information, unpublished assessment material, or answer keys. The UvA environment provides stronger data protection than commercial chat tools, but careful data minimisation is still required.
- **Responsibility:** plausible output can still be wrong, biased, or based on invented facts. You remain responsible for checking and correcting it.

Submitting generated text, code, or a model as if it were entirely your own work may be treated as plagiarism or fraud. Unless a teacher explicitly says otherwise, use UvA AI Chat rather than a commercial generative-AI service for course work.

What is the Socratic method?

The [Socratic method](#) is a question-led dialogue for examining an idea. Instead of immediately supplying an answer, the questioner asks the learner to state a claim, clarify its assumptions, justify each step, and reconsider the claim when a contradiction or counterexample appears. The learner therefore reaches or revises a conclusion through their own reasoning.

In this course, asking UvA AI Chat to work Socratically means asking it to pose one focused question at a time, request evidence, and offer hints or counterexamples before showing a complete solution. The purpose is to make your reasoning visible and testable, not simply to delay the answer.

Set up the ABW Socratic Coach

A persona stores instructions for how UvA AI Chat should conduct a conversation. Create a persona using the text below. If the teaching team shares an official version, use **Copy Items** to make your own copy and check that its version matches the one shown here. A copied persona can be adapted, so keep the original course instructions intact when experimenting with your own version.

UvA AI Chat reusable persona

ABW Socratic Coach (version 2026.08)

You are the ABW Socratic Coach. Your purpose is to develop my reasoning, not to do assessed work for me. At the start, ask which mode I need—Concept coach, Code reviewer, or Model auditor—and ask me to show my own attempt. Ask one focused question at a time and wait for my answer. Progress from a diagnostic question to a small hint, a counterexample, and a partial worked step; give a complete solution only after I have made a serious attempt, explicitly request it, and confirm that the course rules allow it. For code, ask me to predict output, dimensions, indices, data types, and boundary cases, then require a test before suggesting a correction. For an optimization model, ask for sets, parameters, variables, units, objective, and constraints, then look for the smallest feasible counterexample to my formulation. Treat every result as unverified. End by asking me to state what I changed, what evidence I checked, and what uncertainty remains. If I say that the task is an exam or AI-prohibited graded work, do not solve it; switch to general conceptual practice. Never ask me to upload personal or confidential data, unpublished assessment material, or answer keys.

If the persona is unavailable, paste these persona instructions before using a weekly blue prompt. For a clean exercise, start a new chat and review or disable **Memory** when you do not want details from earlier conversations to influence the coaching. A persona controls the dialogue; it does not make an answer correct.

The ABW verification loop

Use this five-step loop whenever AI support is allowed:

1. **Attempt.** Write down your own reasoning, model, or code first.
2. **Ask for coaching.** Use the ABW Socratic Coach and select the mode that fits the task. Ask for one hint, counterexample, or diagnostic question before asking for a complete solution.
3. **Test.** Run the code. Check data types, dimensions, units, feasibility, constraints, boundary cases, and the objective value.
4. **Explain.** Rewrite the result in your own words and connect it to the relevant ABW concept. If you cannot explain it, you have not finished.
5. **Record.** When reporting is required, keep the relevant prompt, output, corrections, and a short statement of what you accepted or rejected.

Three reusable persona modes

Use these short prompts after selecting the ABW Socratic Coach. Without the persona, paste its full instructions above first.

UvA AI Chat study prompt

Concept-coach mode

Mode: Concept coach. I am learning [concept]. Begin by asking what I currently believe and why. Use the notation from the exercise, probe one assumption at a time, and finish by asking me to explain the concept in my own words and give one example and one counterexample.

UvA AI Chat study prompt**Code-reviewer mode**

Mode: Code reviewer. Review my Python or Pyomo code without rewriting it first. Identify the smallest plausible cause of the problem and ask me to run one test that could confirm or reject it. Focus on index sets, dimensions, data types, and boundary cases.

UvA AI Chat study prompt**Model-auditor mode**

Mode: Model auditor. Try to falsify my formulation. Inspect missing indices, inequality directions, units, bounds, and feasible solutions that violate an intended business rule. Ask me to construct the smallest counterexample before discussing a correction.

What counts as verification?

An AI response is not evidence by itself. Useful evidence includes a reproducible code run, a hand calculation on a small instance, comparison with a known result, an independently checked source, a feasibility check, or a counterexample. For an optimization model, at minimum inspect the solver status, objective value, important decision variables, and whether every business rule is satisfied by the returned solution.

Official UvA guidance

The current rules and feature descriptions can change. Consult the [UvA AI Chat student page](#), the [general UvA guidance on AI in your studies](#), and the latest user guide available through the question-mark menu in UvA AI Chat. These UvA-wide rules always apply in addition to the instructions for this course and for each assessment.

The course story

Notebooks are maintained in the public [ABW collection](#) and [shared foundations](#) on GitHub. Public notebook links open these GitHub sources in Colab. [Canvas for ABW 2026/27](#) remains the place for course announcements, deadlines and released solutions. Private instructor sources are not student downloads; request a released copy if needed. The 2026/27 materials are still being developed.

ABW is not a collection of unrelated Python tricks and optimization formulas. The six tutorials follow one analytical storyline: represent a real problem precisely, compute something useful, challenge the result, and explain why it can be trusted.

Weeks 1–2: computation and data

You first learn to express a line of reasoning as executable Python and then to organise data so that the same reasoning can be applied reliably and repeatedly.

Week 3: patterns and paths

Machine learning asks what can be inferred from observed data. Shortest-path algorithms ask which decision is best on a known network. Putting them beside one another makes the distinction between prediction and optimization explicit.

Weeks 4–5: decisions as models

Linear optimization turns a decision problem into variables, an objective, and constraints. Indexed notation then connects that mathematical model to structured data without losing the meaning of the business rules.

Week 6: discrete decisions

Mixed-integer models express choices that cannot be divided continuously. The final step is to understand both the expressive power and the computational cost of integrality.

Across all six weeks, return to four questions:

1. What is the object being represented: a computation, dataset, prediction, network, or decision?
2. Which assumptions and units give the representation its meaning?
3. What small example, boundary case, or counterexample could expose an error?
4. What evidence would convince another analyst that the result is correct and useful?

Each week begins with an optional UvA AI Chat study prompt. The prompts deliberately ask the tool to question, challenge, and request evidence rather than produce an answer. They are intended for permitted tutorial practice; the rules for graded work and assessments remain those stated in the preceding section.

1 First week: Wednesday, September 2, 2026

UvA AI Chat study prompt

Trace before you run

Use the ABW Socratic Coach persona (version 2026.08). If it is unavailable, first paste the persona instructions from the course guide. Mode: Code reviewer. I will show you a short program and my predicted output. Make me trace it one line at a time and design one boundary case that could disprove my prediction. Finish by asking what evidence confirms the result.

We assume that you understand the contents of the notebook [Python summary](#).

This week we exercise Python expressions, variables, types and simple code.

Please use the following additional notebook for self-learning a [gentle introduction to Python, part 1](#) before you try to answer these questions.

You may also like to experiment with the code of the assignments in your own notebook.

Keep in mind that the exam is invigilated and completed entirely with pen and paper. The only permitted device is a UvA-approved calculator with no connectivity. You must therefore be able to reason through a similar question without a computer and show your reasoning on paper.

Note that [Nicole Guarnieri](#), tutorial teacher coordinator, created a structured preparation notebook while preparing herself to teach the first week's exercises last year.

Following this example and creating structured notebooks that combine your notes with the different sources leads to a very effective way of studying and learning.

Instructor reference: [Nicole's historical preparation notebook](#) (*instructor access*).

Exercise 1.1

Suppose you have the following code:

```
if x <= 2:
    if y >= 3:
        z = 3
    else:
        z = 4
elif x >= 3:
    if y <= 2:
        z = 3
    else:
        z = 9
```

- a) If `x = 1` and `y = 2`, what is `z`?
- b) `z` can be 3, 4 or 9, what values can `x` and `y` be if you want to have `z = 4`?

Solution 1.1

- a) `z = 4`
- b) `x` should be less or equal to 2, and `y` should be less than 3 (not equal).

Exercise 1.2

What will be printed in the following cases:

a)

```
print(2 + 'a')
```

b)

```
print(2 * 'a')
```

c)

```
print('2' + 'a')
```

Solution 1.2

- a) Will result in `TypeError: unsupported operand type(s) for +: 'int' and 'str'`
- b) `aa`
- c) `2a`

Exercise 1.3

- a) What are the four primitive types used in Python?
- b) Explain, using your own words, what the difference is between primitive and non-primitive types, sometimes called data structures?
- c) Provide an example of a situation in which a dictionary is more useful than a list. Why?

Solution 1.3

- a) Boolean, Integer, Float and String
- b) Non-primitive data structures exist of collections of other primitive and non-primitive data structures, while primitive data types serve as the “raw material” itself.
- c) A dictionary is more useful than a list when: there is no clear natural ordering of the data ($0, \dots, n$ indexing would not make sense) and/or when the data has unique id's with some associated meaning.

An example of a dictionary could be the storage of statistics per football club in Europe, where the football club names could serve as the ‘key’ and their respective statistics as the ‘value’.

Provided that the type used to identify each entry of the dictionary (the so-called key) is an immutable object (means that its value will not change unless explicitly stated) and has the property of being hashable (means that a function exists that translates each value of this key to a not too big numerical index, and that this translation is very often unique, see https://en.wikipedia.org/wiki/Hash_function) then dictionaries treat these keys in the same way as the indices in a list. Even if the keys are numbers that could be indices, this allows for sparsity (suppose that you only have a few keys but they are numbers of six digits: using them to index a list would waste a lot of space since all lists start at 0).

Exercise 1.4

What do you expect as a result of the following:

```
[ x for x in range(20) if x % 3 == 0 ]
```

Solution 1.4

The multiples of 3 in the range from 0 to 20 (where 20 is not inclusive): [0, 3, 6, 9, 12, 15, 18].

Exercise 1.5

Consider the following code:

```
'-'.join('Hello world'.split(' '))
```

- a) What is the result of `split` ?
- b) What is the final result?

Solution 1.5

- a) Calling `split` from a string yields a list of substrings separated by the argument, in this case a space `' '`. Therefore it yields `['Hello', 'world']`.
- b) Calling `join` from a string on a list of strings yields a string that is the concatenation of the elements of the list, joined by the given string, in this case `'Hello-world'`. The same result could be achieved by using the `replace` function, see https://www.tutorialspoint.com/python/string_replace.htm for more information.

Exercise 1.6

The code below creates a dictionary, using comprehension.

```
{ x : x**3 for x in range(3) }
```

Can you describe the keys and values of that dictionary? Please comment on this use of a dictionary.

Solution 1.6

The keys are the numbers in `range(3)`, i.e. `[0, 1, 2]`. The values are the corresponding cubes. Therefore, the dictionary maps 0 to 0, 1 to 1 and 2 to 8. Asking to comment is always an open question... one comment could be that this could just be a list, since the dictionary keys are exactly a range starting at 0. In fact, reversing the keys with the values of this dictionary would make it more useful!

Exercise 1.7

Whats wrong in the code below?

```
my_list = ['a', 'b']  
my_list[2] = 'c'
```

Can you fix it to achieve the desired result?

Solution 1.7

The attempt to assign `'c'` to the position 2 of the list fails (gives an error), as the list has no position 2 to assign to! The intention seems to add `'c'` to the list, which is achieved by doing `my_list.append('c')`.

Exercise 1.8

- a) The code below creates an integer, assigns it to variable `x`, and assigns `x` to a new variable `y`. Then an alteration is made to `x` and `y` is printed. What will be printed and why?

```
x = 1
y = x
x = 10
print(y)
```

- b) The code below creates a list and assigns it to variable `x`, assigns `x` to a new variable `y`, then an alteration is made and `y` is printed. What will be printed and why?

```
x = [1, 2, 3]
y = x
x[0] = 10
print(y)
```

- c) Consider the code presented above in question b). Is there a way to prevent variable `y` from changing when variable `x` is altered? How can this be done?

Solution 1.8

- a) The integer `1` will be printed. An int is a so called “immutable” object in Python. The variable `y` is assigned to the int 1 (irrespective of variable `x`), so when `x` is altered, this alteration will not be reflected in `y` (which simply holds the value 1).
- b) The list `[10, 2, 3]` will be printed. A list is a so called “mutable” object in Python. The variable `y` points to the same list object assigned to the variable `x`, thus when the list object itself is altered, this alteration will be reflected in all variables that point to the list object.
- c) It is certainly possible to prevent this. Instead of setting `y = x`, we could set `y = x.copy()` or `y = list(x)` or even `y = x[:]` to create a copy of the `[1, 2, 3]` list object. If interested, visit <https://docs.python.org/3/library/copy.html> for more information on this topic.

Exercise 1.9

The code below has a serious problem.

```
x = 0
while x != 2:
    print(x)
    x += .1
```

Can you spot it? Hint: look at <https://stackoverflow.com/questions/588004/is-floating-point-math-broken>.

Solution 1.9

This may be a hard question, since it relates to the imperfect representation of decimals in a computer. Please also read <https://0.30000000000000004.com/>. The code runs for ever, because `x` never becomes exactly 2. Try it!!! The way to fix it is replacing the condition that guards the `while` by `x < 2`. This code will terminate as intended:

```
x = 0
while x < 2:
    print(x)
    x += .1
```

Exercise 1.10

Slicing lists amounts to doing `my_list[start:stop:step]`. <https://stackoverflow.com/questions/509211/understanding-slice-notation> gives a great explanation. After studying that explanation please say what the code below should yield.

- | | | | |
|------------------------------|-----------------------------|-----------------------------|-------------------------------|
| a) <code>'ABW'[0:3:1]</code> | e) <code>'ABW'[:−1:]</code> | i) <code>'ABW'[1:]</code> | m) <code>'ABW'[:3:1]</code> |
| b) <code>'ABW'[:,:]</code> | f) <code>'ABW'[:−1]</code> | j) <code>'ABW'[:1]</code> | n) <code>'ABW'[:2021]</code> |
| c) <code>'ABW'[:]</code> | g) <code>'ABW'[−1]</code> | k) <code>'ABW'[1::3]</code> | o) <code>'ABW'[:2021:]</code> |
| d) <code>'ABW'[:,−1]</code> | h) <code>'ABW'[1]</code> | l) <code>'ABW'[3::1]</code> | p) <code>'ABW'[2021:]</code> |

Solution 1.10

- | | | | |
|-----------------------|----------------------|----------------------|-----------------------|
| a) <code>'ABW'</code> | e) <code>'AB'</code> | i) <code>'BW'</code> | m) <code>'ABW'</code> |
| b) <code>'ABW'</code> | f) <code>'AB'</code> | j) <code>'A'</code> | n) <code>'ABW'</code> |
| c) <code>'ABW'</code> | g) <code>'W'</code> | k) <code>'B'</code> | o) <code>'ABW'</code> |
| d) <code>'WBA'</code> | h) <code>'B'</code> | l) <code>'</code> | p) <code>'</code> |